

LLM-Augmented Renaming of Publications

Overview

This repository helps normalize, rename, and prepare ebook and publication files for downstream indexing or RAG ingestion.

It supports two main rename flows:

1. `rename-using-llm.sh` Renames files from their content using an LLM API.
2. `rename-using-ebooks-tools.sh` Uses `ebook-tools` and related scripts for metadata-based renaming.

The preferred end-to-end entry point is:

```
./rename.sh /path/to/books
```

That wrapper:

1. runs `rename-using-llm.sh`
2. then converts any remaining non-PDF renamed files to PDF using the conversion scripts in this repo

Supported source formats in the main LLM flow: `pdf`, `epub`, `mobi`, `chm`

Current File Behavior

The repo no longer writes renamed files into a `Renamed/` subdirectory.

Current behavior for the LLM-based flow:

- renamed files stay in the same directory as the original file
- the pre-rename source file is copied into a sibling `Originals/` directory
- files that cannot be processed are moved into `Failed/`
- `mobi` and `chm` files are converted to PDF as part of the rename step

Current behavior for the conversion scripts:

- converted PDFs are written into the same directory as the source file
- the source non-PDF file is moved into a `Converted/` subdirectory after successful conversion

Current behavior for `rename-using-ebooks-tools.sh`:

- Docker output is staged temporarily
- final renamed files are moved back into the input directory
- original top-level input files are copied into `Originals/`

Repository Layout

- `rename.sh` Preferred wrapper for LLM rename plus post-rename PDF conversion.
- `rename-using-llm.sh` Content-based rename flow using an LLM endpoint.
- `rename-using-ebooks-tools.sh` Metadata-based rename flow using `ebook-tools`.
- `fix-matches.sh` Repairs the directory structure produced by `ebook-tools`.
- `convert-epub-to-pdf.sh`
- `convert-mobi-to-pdf.sh`
- `convert-chm-to-pdf.sh`
- `convert-azw3-to-pdf.sh` One-format conversion helpers.

Installation

Clone the repository:

```
git clone https://github.com/ngpepin/LLM-rename-publications-RAG.git
cd rename-ebooks
```

Install the main dependencies:

- `jq`
- `docker`
- `unzip`
- `poppler-utils`
- `calibre`

Depending on which scripts you use, you may also need:

- `mobi_unpack`
- `file`

Configuration

The main configuration files are:

- `rename-using-llm.conf`
- `rename-using-ebooks-tools.conf`
- `config.json`

`rename-using-llm.sh` expects a working API endpoint and model configuration in `rename-using-llm.conf`.

Usage

Preferred Wrapper

```
./rename.sh /path/to/books
./rename.sh --llm /path/to/books
./rename.sh --ebook-tools /path/to/books
```

Use this when you want:

- LLM-based semantic renaming first by default
- then conversion of renamed **epub**, **mobi**, **chm**, and **azw3** files to PDF

Use **--ebook-tools** if you want the metadata-based rename flow before the same conversion pass.

The wrapper skips **Originals/**, **Failed/**, and **Converted/** directories during its conversion pass.

LLM-Based Renaming Only

```
./rename-using-llm.sh /path/to/books
```

Use this when you want semantic renaming without the extra conversion pass performed by **rename.sh**.

Metadata-Based Renaming

```
./rename-using-ebooks-tools.sh -i /path/to/input -o /path/to/output
```

Or, with a single directory argument:

```
./rename-using-ebooks-tools.sh /path/to/input
```

In the current implementation, final renamed files are placed back into the input directory and originals are archived into **Originals/**.

Individual Conversion Scripts

```
./convert-epub-to-pdf.sh /path/to/books
./convert-mobi-to-pdf.sh /path/to/books
./convert-chm-to-pdf.sh /path/to/books
./convert-azw3-to-pdf.sh /path/to/books
```

These scripts operate on files in the specified directory only (**maxdepth 1**).

RAG-Oriented Workflow

Typical usage looks like this:

1. Normalize and rename source documents.

```
./rename.sh /data/publications
```

2. Feed the resulting PDFs and archived originals into your chunking, embedding, and indexing pipeline.

Notes

- Re-running the LLM flow is intended to be safe because processed originals are archived and collisions are handled.
- Conversion scripts create **Converted/** directories only when they successfully move source files out of the working directory.
- Logs are written under **logs/**.

License

MIT License.